

Neural Network implementing Universal Approximation Theorem from Scratch Using Only NumPy

Rajwardhan Dasture

June 17, 2026

1 Introduction

This project aims in making a neural network which is made from scratch learn $\sin(x)$ with no knowledge of trigonometry prior to it using UAT (Universal Approximation Theorem).

The entire Neural Network is built using NumPy just Mathematics. Libraries such as Pytorch and TensorFlow are not used even once. The autograd engine, the layers, the optimizer, and the training loop are all implemented ground up. The goal was to understand exactly why every piece works, and to verify that understanding against numerical checks and PyTorch. This project took about 2 months to completely understand and execute.

Also notable is the fact that the broadcasting bug in gradient computation (Section 4), the SGD instability at epoch 1000 that Adam silently fixed, and the fact that 8 neurons can approximate $\sin(x)$ better than 128 under a fixed training budget — which is a reminder that the UAT is an existence result, not an optimization guarantee. All the program files and images of the results are attached along with it.

2 The Universal Approximation Theorem

Definition 1 (Feedforward Neural Network). *A single-hidden-layer network with n neurons is a function $F : \mathbb{R}^d \rightarrow \mathbb{R}$ of the form*

$$F(\mathbf{x}) = \sum_{j=1}^n \alpha_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j), \quad (1)$$

where $\mathbf{w}_j \in \mathbb{R}^d$, $b_j, \alpha_j \in \mathbb{R}$, and σ is the activation function.

Theorem 1 (Cybenko 1989, Hornik 1991). *Let σ be any continuous sigmoidal function and $K \subset \mathbb{R}^d$ compact. For every $f \in C(K)$ and every $\varepsilon > 0$, there exist $n \in \mathbb{N}$ and parameters $\{\mathbf{w}_j, b_j, \alpha_j\}$ such that*

$$\sup_{\mathbf{x} \in K} \left| f(\mathbf{x}) - \sum_{j=1}^n \alpha_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j) \right| < \varepsilon. \quad (2)$$

Proof Sketch. Suppose the set $\mathcal{H}_\sigma$ of all functions of the form (1) is *not* dense in $C(K)$. Then its closure is a proper closed subspace, and by the Hahn-Banach theorem there exists a non-zero bounded linear functional L on $C(K)$ with $L(h) = 0$ for all $h \in \mathcal{H}_\sigma$. By the Riesz representation theorem, L corresponds to a signed Borel measure μ satisfying

$$\int_K \sigma(\mathbf{w}^\top \mathbf{x} + b) d\mu(\mathbf{x}) = 0 \quad \forall \mathbf{w}, b. \quad (3)$$

For a sigmoidal σ , this integral transform is injective — if it vanishes at every frequency, then $\mu \equiv 0$, which forces $L \equiv 0$, a contradiction. Therefore $\mathcal{H}_\sigma$ is dense in $C(K)$. $\square$

Remark 1. *This is an existence result only. It guarantees that weights achieving ε -approximation exist — it says nothing about how to find them. Gradient descent is the practical mechanism.*

Corollary 2. *Hornik (1991) extended this to any non-polynomial continuous activation, including ReLU and tanh.*

3 Backpropagation Derivations

3.1 Forward Pass

For a network with L layers, let $\mathbf{a}^{(0)} = \mathbf{x}$. For $\ell = 1, \dots, L$:

$$\mathbf{z}^{(\ell)} = W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad (4)$$

$$\mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)}). \quad (5)$$

Loss (MSE over N samples):

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left\| \mathbf{a}_i^{(L)} - \mathbf{y}_i \right\|^2. \quad (6)$$

3.2 Backward Pass

Define the error signal $\boldsymbol{\delta}^{(\ell)} = \partial \mathcal{L} / \partial \mathbf{z}^{(\ell)}$.

Output layer:

$$\boldsymbol{\delta}^{(L)} = \frac{2}{N} (\mathbf{a}^{(L)} - \mathbf{y}) \odot \sigma'(\mathbf{z}^{(L)}). \quad (7)$$

Hidden layers:

$$\boldsymbol{\delta}^{(\ell)} = \left(W^{(\ell+1)\top} \boldsymbol{\delta}^{(\ell+1)} \right) \odot \sigma'(\mathbf{z}^{(\ell)}). \quad (8)$$

Parameter gradients:

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \mathbf{a}^{(\ell-1)\top}, \quad (9)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)}. \quad (10)$$

3.3 Activation Derivatives

$$\tanh'(z) = 1 - \tanh^2(z), \quad (11)$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \quad (12)$$

$$\text{ReLU}'(z) = \mathbf{1}[z > 0]. \quad (13)$$

3.4 Xavier Initialization

To keep activation variance stable across layers (Glorot & Bengio, 2010):

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, +\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right). \quad (14)$$

Derived by requiring $\text{Var}(z^{(\ell)}) \approx \text{Var}(z^{(\ell-1)})$ under symmetric zero-mean weights.

4 Automatic Differentiation Engine

4.1 Design

The `Tensor` class wraps a NumPy array and maintains a `_backward` closure and a set of parent nodes `_prev`. Calling `.backward()` on the loss triggers a topological sort of the computation graph and propagates gradients in reverse:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} += \text{._backward}\left(\frac{\partial \mathcal{L}}{\partial \text{out}}\right). \quad (15)$$

The `+=` accumulation handles nodes with multiple downstream consumers.

4.2 Broadcasting and Gradient Reduction

The most subtle bug encountered: when bias $\mathbf{b}$ of shape $(m,)$ is broadcast to (N, m) in the forward pass, the upstream gradient has shape (N, m) but `$\mathbf{b}.\text{grad}$` has shape $(m,)$. The fix is to sum over the broadcast dimension:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial z_i}, \quad (16)$$

by the chain rule: $\mathbf{b}$ influences $\mathcal{L}$ through N paths, so its total gradient is the sum of all N upstream gradients. This applies to every binary operation that admits broadcasting.

4.3 Gradient Verification

Two independent checks were run:

Numerical gradient check (central difference, $h = 10^{-5}$):

$$\nabla_{\theta}\mathcal{L} \approx \frac{\mathcal{L}(\theta + h) - \mathcal{L}(\theta - h)}{2h}. \quad (17)$$

All weight gradients matched to relative error $< 10^{-10}$; bias gradients to $< 10^{-3}$ (floating-point noise at this scale of h).

PyTorch comparison: identical weights and inputs passed through both engines. All gradients agreed to $< 3 \times 10^{-8}$ absolute difference — consistent with float64 rounding.

5 Experiments

5.1 Setup

Domain: $x \in [-\pi, \pi]$, 200 uniformly spaced points. Architecture: Dense($1 \rightarrow 64$) $\rightarrow$ tanh $\rightarrow$ Dense($64 \rightarrow 64$) $\rightarrow$ tanh $\rightarrow$ Dense($64 \rightarrow 1$). Optimizer: Adam, $\eta = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, 3000 epochs.

5.2 Four Target Functions

$$f_1(x) = \sin(x), \quad (18)$$

$$f_2(x) = x \sin(x), \quad (19)$$

$$f_3(x) = x \sin(x) + \cos(x^2), \quad (20)$$

$$f_4(x) = \sin(3x) e^{-0.1x^2} + 0.1x. \quad (21)$$

Each is continuous on $[-\pi, \pi]$ and therefore UAT-approximable. The same architecture was used for all four — only the weights differ.

Table 1: Final MSE after 3000 epochs.

	Function	Final MSE
f_1	$\sin(x)$	2.29×10^{-5}
f_2	$x \sin(x)$	2.55×10^{-5}
f_3	$x \sin(x) + \cos(x^2)$	1.27×10^{-4}
f_4	$\sin(3x)e^{-0.1x^2} + 0.1x$	7.26×10^{-6}

5.3 Width Study

To probe the UAT’s width dependence, f_1 was approximated with $n \in \{2, 4, 8, 16, 32, 64, 128\}$ neurons:

Table 2: Approximation error vs neuron count ($f_1 = \sin x$, 2000 epochs).

Neurons	Final MSE
2	1.13×10^{-2}
4	1.45×10^{-5}
8	1.80×10^{-6}
16	9.92×10^{-6}
32	1.26×10^{-4}
64	7.72×10^{-5}
128	7.49×10^{-5}

The non-monotonic result ($n = 8$ outperforming $n = 128$) is not a failure of the UAT — larger networks have more complex loss landscapes and Adam does not always find as good a minimum under a fixed training budget. The theorem guarantees existence of good weights, not that any optimizer finds them.

5.4 SGD vs Adam

Training f_1 with plain SGD ($\text{lr} = 0.05$) produced a loss spike at epoch 1000: loss jumped from 3.7×10^{-3} to 2.5×10^{-1} before recovering. This is fixed-step-size overshooting in a narrow loss-surface ravine. Adam’s first moment estimate smooths gradient oscillations and eliminated the spike entirely, converging to $\mathcal{L} < 10^{-6}$ by epoch 1800.

6 Loss Landscape

The trained network’s loss surface was visualized using the filter-normalization method . Two random directions $\mathbf{d}_1, \mathbf{d}_2$ in parameter space are normalized layer-wise to match trained weight norms:

$$\mathbf{d}_i^{(\ell)} \leftarrow \frac{\mathbf{d}_i^{(\ell)}}{\|\mathbf{d}_i^{(\ell)}\|} \cdot \|\theta^{(\ell)}\|. \quad (22)$$

The surface $\mathcal{L}(\theta^* + \alpha \mathbf{d}_1 + \beta \mathbf{d}_2)$ was evaluated on a 30×30 grid with $\alpha, \beta \in [-1, 1]$. The result shows a wide smooth basin (loss = 6×10^{-6} at center, rising to 37.3 at edges) with asymmetric curvature along the two directions — reflecting unequal Hessian eigenvalues along $\mathbf{d}_1$ and $\mathbf{d}_2$.

7 Conclusion

A feedforward neural network, built using only NumPy, approximated four continuous functions to MSE below 1.3×10^{-4} which directly demonstrates the applicability the Universal Approximation Theorem experimentally. Gradient correctness was verified numerically and against PyTorch. Every component including autograd, Xavier init, broadcasting-aware gradients and Adam were derived from first principles and confirmed to produce results identical to a production framework.

8 Code Structure

Listing 1: Project layout

```
1 NN_for_UAT/  
2   engine/  
3       tensor.py      # Tensor class, autograd graph, backward()  
4       ops.py         # Pure NumPy forward/backward primitives  
5   layers/  
6       base.py        # Abstract Layer base class  
7       dense.py       # Fully connected layer, Xavier init  
8       activations.py  # ReLU, Sigmoid, Tanh  
9       loss.py        # MSE loss  
10  network/  
11      model.py        # Sequential container  
12      optimizer.py    # SGD and Adam  
13  experiments/  
14      01_simple_sin.py    # sin(x) approximation + Adam vs SGD  
15      02_width_study.py   # UAT width dependence  
16      03_hard_function.py  # Four target functions  
17  verification/  
18      gradient_check.py    # Numerical gradient verification  
19      compare_pytorch.py   # PyTorch agreement check  
20  visualization/  
21      loss_landscape_3d.py  # Filter-normalized loss surface  
22  proofs/  
23      main.tex            # This document
```